

PITBULL: Temporal Safety Skill for Coding Agents

Pavel Marian¹, Stanislav Chumakov¹, Danil Ezhov¹, Alexander Boukhanovsky¹

¹AI Institute, ITMO University, Saint Petersburg, Russian Federation

pmarian@itmo.ru, svchumakov@itmo.ru, doezhov@itmo.ru, boukhanovsky@mail.ifmo.ru

Abstract

We present PITBULL, an executable temporal-safety skill for agent-generated feature programs. The skill detects point-in-time violations by comparing pipeline outputs on full and temporally truncated databases, adds canary perturbations for late-arriving fields, and validates every agent repair on held-out prediction times. Across three runs over 62 leaking programs, 97.8% of program-run pairs were repaired within five iterations. Packaged as a skill that an independent coding agent selects on its own, the same loop repairs 76 of 76 programs, matching direct invocation of the procedure.

Demo — <https://pavelmarian.github.io/pitbull/demo/>

Code — <https://github.com/pavelmarian/pitbull>

Video — <https://youtu.be/ST8dWf82Zxs>

Introduction

Successful execution of a relational feature pipeline by a coding agent does not guarantee its temporal correctness. A pipeline that filters on a single reference timestamp can still read information that did not exist at the prediction time, because availability is spread across related tables and columns. Such violations yield plausible features and inflated offline results that do not reproduce at serving time. The property that every feature computed at time t depends only on facts available at time t is called point-in-time correctness.

The model alone misses most leaks: in our pilot, asked to check its own code for errors, it found the leak in 1 of 12 programs, and given the class of error by name it repaired 11 of 12. Reading code, it flags leaking programs with 97% recall and 12% false positives, which makes LLM inspection a signal and leaves the gate to execution. PITBULL closes this gap with a loop of execution-based detection, agent repair, and independent revalidation, packaged as a skill.

Related Work

Temporal leakage is a recurring source of irreproducible machine-learning results (Kaufman et al. 2012; Kapoor and Narayanan 2022), including in temporally split process-mining benchmarks (Weytjens and Weerd 2021). In relational data, features span several tables (Robinson et al. 2024)

and a value’s availability may be governed by another row or by a column-specific timestamp. Static notebook analysis and pipeline instrumentation target preprocessing and train-test contamination (Yang et al. 2022; Grafberger et al. 2021); LLM-based detectors read the code (Alturayef and Hassine 2025). The PITFALL system introduced black-box differential execution for detecting and localising point-in-time violations (Chumakov 2025); PITBULL extends it from diagnosis to repair. The verification is one-sided, as in the formulation of look-ahead freedom as temporal non-interference: a divergence is a finite witness of a violation, while agreement at finitely many probes leaves the program unverified (Fonseca 2026).

System Overview

PITBULL is a skill, a directory with a `SKILL.md` instruction and scripts that a coding agent loads by its description (the format of Claude Code and OpenHarness). The agent invokes it over a candidate feature program $\varphi(D, e, t)$: a database D , entities e , and a prediction time t . The agent writes and revises the program; the skill only executes it and returns evidence (Figure 1). Which column dates each table is declared in a hand-written availability map; the check is only as good as that map.

Differential witness. For each prediction time the skill runs the program twice, on the full database and on the database truncated at t :

$$y_{\text{full}} = \varphi(D, e, t), \quad y_{\text{past}} = \varphi(D|_t, e, t). \quad (1)$$

Any difference beyond a numerical tolerance is a LEAK witness, and the skill names the affected output columns. Equality only says that no violation was observed at the times tested.

Canary. Some channels survive truncation: a delivery date sits in a row placed before t and is filled in later. The skill therefore rewrites such late-filled fields and all future rows with marker values (shifted numbers, a reserved category, dates beyond the data horizon). An output that reacts is a canary hit. Canary fired on 4 of 30 clean control programs, so its columns are listed separately for a human to judge.

Repair loop and held-out gate. On a LEAK, the agent rewrites the program under a fixed natural-language instruction that names the class of error (features must use only

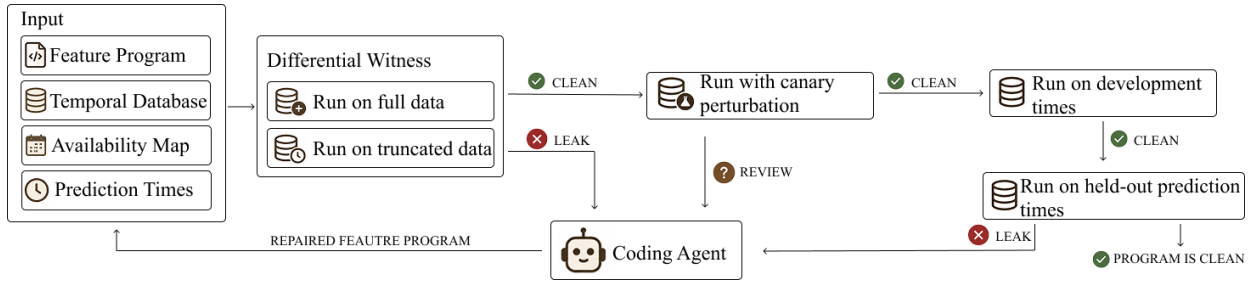


Figure 1: One round of the PITBULL loop. The program runs on three views of the same database; a feature that changes between them was computed from data that did not exist at `seed_time`. The agent receives the differing feature and its source, rewrites the program, and the candidate is re-checked at six prediction times, three of them never shown to the agent.

information that exists at t ; reviews and delivery dates arrive after the purchase) and reruns the check. A candidate is **CLEAN** only if it executes and shows neither witness nor canary divergence on all development and held-out prediction times; missing code, runtime errors and timeouts count as failures. The loop stops at the first **CLEAN** or after five repairs. The full history of candidates and their verdicts is kept in a state file next to the code. A **CLEAN** verdict is scoped to the declared availability map and the tested times.

Demonstration

The demonstration is a browser page at the linked address. It opens with the loop of Figure 1 and one worked run: a leaking program from the corpus, the three database views, the diverging feature and its source (reviews joined without a cut on their creation date), the agent’s repair, and the gate on three development and three held-out prediction times; the patched source, diff and JSON report can be downloaded. Three panels let visitors browse the evidence behind Table 1: **CLEAN@ k** over the 76 programs with the stubborn case explained, the three repeated runs, and detection against the number of prediction times with a clean-control check. The page ends with the findings, the limitations, and two downloads, `SKILL.md` and the 76 leaking programs, for running the check in one’s own agent.

Experimental Snapshot

Corpus: the Olist e-commerce database (four tables) and one task, whether a product is ordered in the 90 days after t . Two code models wrote feature programs from a task description that never mentions leakage; the checker flagged 76 as leaking, 62 by the witness and 14 only by the canary. Sixty leak through reviews, fourteen through delivery dates, two through payments or a variable mix-up. Table 1 summarises repair with one repair model. In three independent direct runs over the 62 witness programs, 182 of 186 program–run pairs became clean within five iterations (cluster bootstrap 95% CI [95.2%, 100%]). Two prediction times give 97.6% conservative witness detection; the witness had no false positives on 30 clean control programs, the canary fired on four.

We then packaged the loop as a skill and ran it under OpenHarness 0.1.4, an agent harness that selects and invokes skills on its own; the skill name does not appear in the task

Invocation	n	CLEAN@1	CLEAN@5	Calls	Cost
Direct, 1 run	76	85.5%	98.7%	91	\$0.12
Direct, 3 runs	62×3	84.4%	97.8%	255	\$0.32
Agent-selected skill	76	98.7%	100%	578	\$0.48

Table 1: Repair outcomes with one repair model (z-ai/glm-5.3-flash). The skill row uses the same 76 programs as the first row; exact McNemar over paired per-program outcomes: 0 vs. 1 discordant, $p = 1.0$.

prompt. On the same 76 programs the skill matches direct invocation: **CLEAN@5** reaches 100%. The one discordant program is the single failure of direct invocation, where the model had added a delivery-date feature that only the canary catches; under the skill it was repaired at the first iteration. In skill mode the agent also sees the checker output with column names, which explains part of the gain at **CLEAN@1**. The price is packaging overhead: about 4× the inference cost and 6× the model calls per program, driven by the harness system prompt on every turn.

Limitations. One database, one task, one repair model, and a corpus dominated by review leakage; a hand-written availability map; a **CLEAN** verdict scoped to six tested times; a canary that flags 13% of clean programs and blocks acceptance; one skill run per program, in which the agent could read the checker’s source.

Conclusion

PITBULL gives a coding agent an executable way to detect, localise, repair, and revalidate point-in-time violations in feature pipelines, with the verdict produced by execution and the repair by the agent. Next: independently annotated per-column availability maps and a corpus beyond review leakage.

References

- Alturayef, N.; and Hassine, J. 2025. Data leakage detection in machine learning code: transfer learning, active learning, or low-shot prompting? *PeerJ Computer Science*, 11: e2730.
- Chumakov, S. 2025. PITFALL. <https://github.com/stas1f1/pitfall>.

- Fonseca, X. 2026. Look-Ahead-Freedom as Temporal Non-Interference: A Verifiable Correctness Property for Backtesting and Agentic Trading Pipelines. arXiv:2607.04958.
- Grafberger, S.; Guha, S.; Stoyanovich, J.; and Schelter, S. 2021. MLINSPECT: A Data Distribution Debugger for Machine Learning Pipelines. In *Proceedings of the 2021 International Conference on Management of Data, SIGMOD '21*, 2736–2739. New York, NY, USA: Association for Computing Machinery. ISBN 9781450383431.
- Kapoor, S.; and Narayanan, A. 2022. Leakage and the Reproducibility Crisis in ML-based Science. arXiv:2207.07048.
- Kaufman, S.; Rosset, S.; Perlich, C.; and Stitelman, O. 2012. Leakage in data mining: Formulation, detection, and avoidance. *ACM Trans. Knowl. Discov. Data*, 6(4).
- Robinson, J.; Ranjan, R.; Hu, W.; Huang, K.; Han, J.; Dobles, A.; Fey, M.; Lenssen, J. E.; Yuan, Y.; Zhang, Z.; He, X.; and Leskovec, J. 2024. RelBench: A Benchmark for Deep Learning on Relational Databases. arXiv:2407.20060.
- Weytjens, H.; and Weerdt, J. D. 2021. Creating Unbiased Public Benchmark Datasets with Data Leakage Prevention for Predictive Process Monitoring. *CoRR*, abs/2107.01905.
- Yang, C.; Brower-Sinning, R. A.; Lewis, G. A.; and Kästner, C. 2022. Data Leakage in Notebooks: Static Detection and Better Processes. arXiv:2209.03345.